

P0595

To: EWG

From: Daveed Vandevoorde (daveed@edg.com)

Date: 2017-02-02

The constexpr Operator

A `constexpr` function may be evaluated at compile time or at run time. However, we may want to vary an implementation depending on which context the function is evaluated in. Specifically, when computing a constant-expression, the algorithm has to use portable, standard C++ appearing in the current translation unit, but run-time computations may want to take advantage of platform-specific facilities and/or separately-compiled libraries.

I propose a new operator invoked as `constexpr()`. It evaluates to `true` when invoked in a context that requires or—for static-lifetime initializers—prefers a constant-expression. It evaluates to `false` when invoked as part of an expression that is not a core constant-expression. In any other context, either `true` or `false` is produced, although the expectation is that `true` will be produced for compile-time evaluations and `false` otherwise (however, that is difficult to mandate because the standard doesn't formally distinguish between compile time and run time).

For example:

```
constexpr double power(double b, int x) {
    if (constexpr() && x >= 0) {
        /* A constant-evaluation context. */
        double r = 1.0, p = b;
        unsigned u = (unsigned)x;
        while (u != 0) {
            if (u & 1) r *= p;
            u /= 2;
            p *= p;
        }
        return r;
    } else {
        /* Let the code generator figure it out. */
        return std::pow(b, (double)x);
    }
}

constexpr double kilo = power(10.0, 3); // (1)
int n = 3;
double mucho = power(10.0, n); // (2)
double thousand() {
    return power(10.0, 3);
}
```

Call (1) occurs in a constant-expression context, and, therefore, `constexpr()` will be `true` during the computation of `power(10.0, 3)`, which in turn allows the evaluation to complete as a constant-expression.

Call (2) isn't a constant-expression because `n` cannot be converted to an rvalue in a constant-expression context. So it will be evaluated in a context where `constexpr()` is `false`; this is known at translation time, and the run-time code generated for the function can therefore easily be reduced to the equivalent of just

```
inline double power'(double b, int x) {
    return std::pow(b, (double)x);
}
```

Call (3) could go either way. The call is technically a constant-expression, but it need not be evaluated as one (it's not called in a context requiring a constant). The expectation is that if the implementation attempts to

evaluate the call at compile time, `constexpr()` will produce `true` and otherwise it will produce `false` (that also falls out of the most likely implementation strategies). In theory, however, a (suspect) implementation could, e.g., evaluate `constexpr()` to `true` while emitting run-time code.

The implementation of this feature is almost trivial: An operator is added in the front end and that operator is always evaluated as `true` within the front end proper (e.g., the `constexpr` interpreter) and as `false` in the stage that emits code to a back end (or middle end).